

IT 2080 – IT Project Activity 04

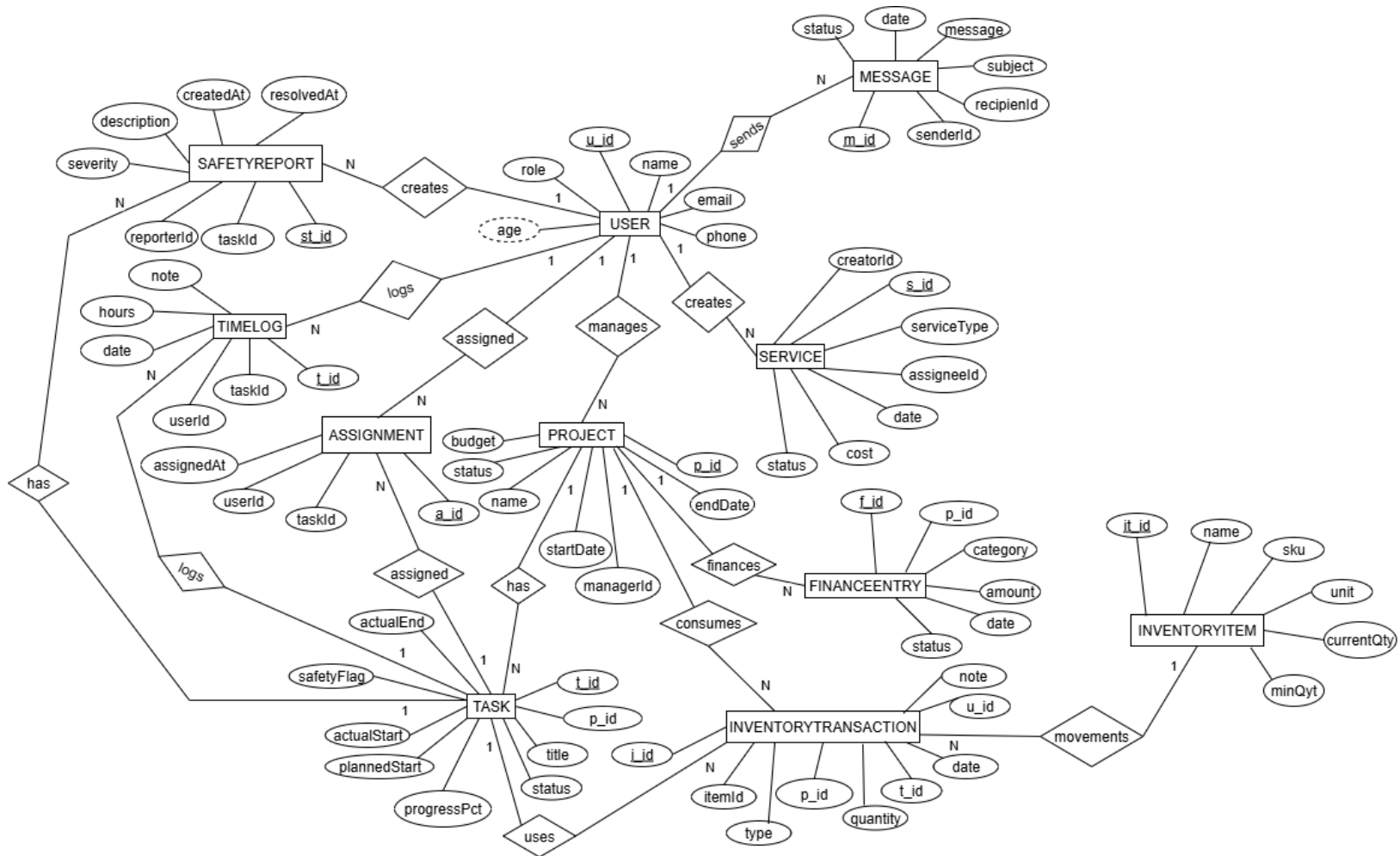


Group Number: ITP_B2_W227

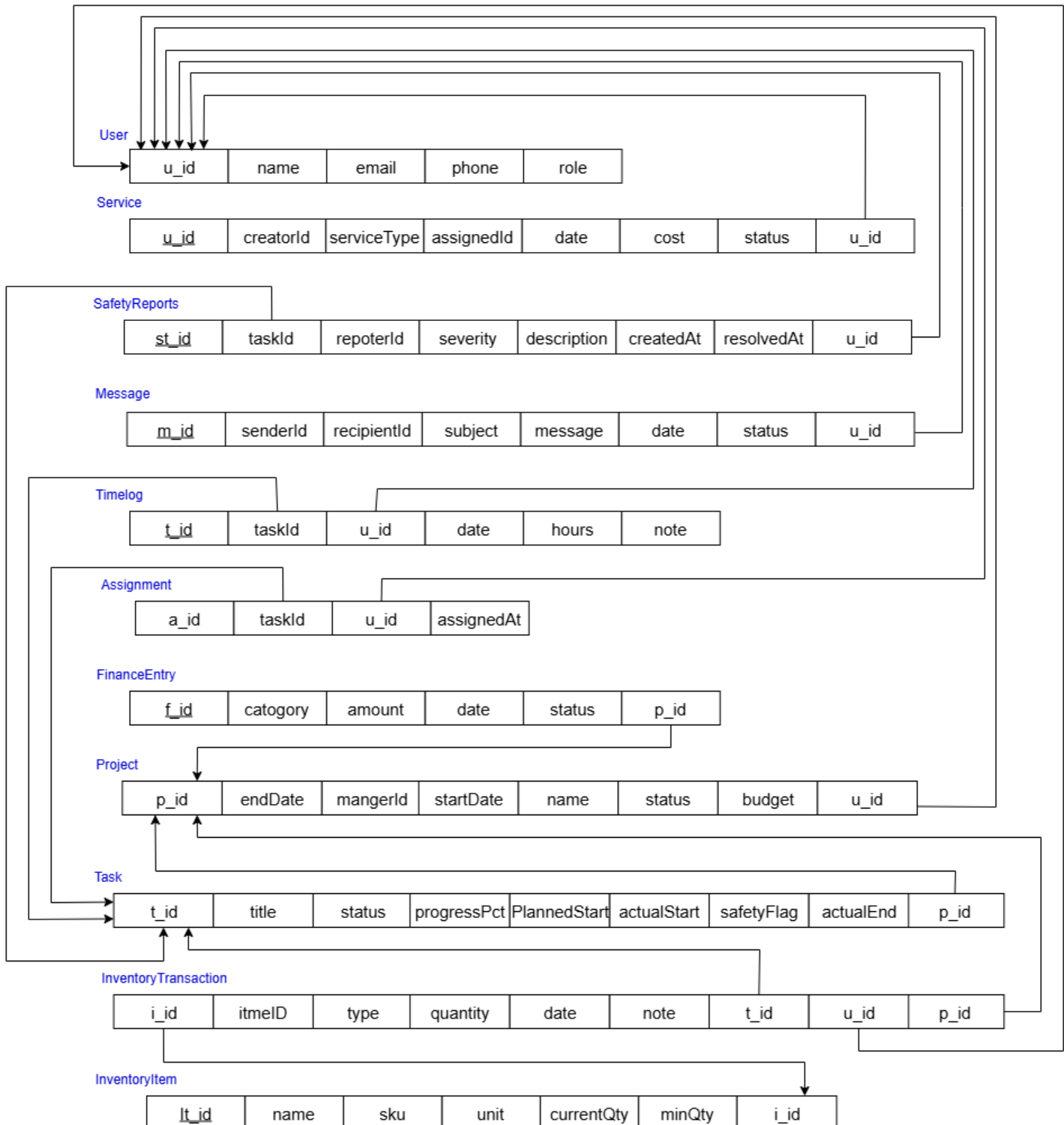
Topic: Construction Management System

	Registration Number	Student Name	Email address	Contact no
1	IT23615502	J.A.A.D. Jayakodi	it23615502@my.sliit.lk	0701038400
2	IT23562424	M.G.C.S. Bandara	it23562424@my.sliit.lk	0787911287
3	IT23698772	D.D.T. Osadara	it23698772@my.sliit.lk	0725556628
4	IT23561816	H.A.B.P. Athukorala	it23561816@my.sliit.lk	0775550410
5	IT23584372	H.G.P.N.Chandrarathna	it23584372@my.sliit.lk	0701622626

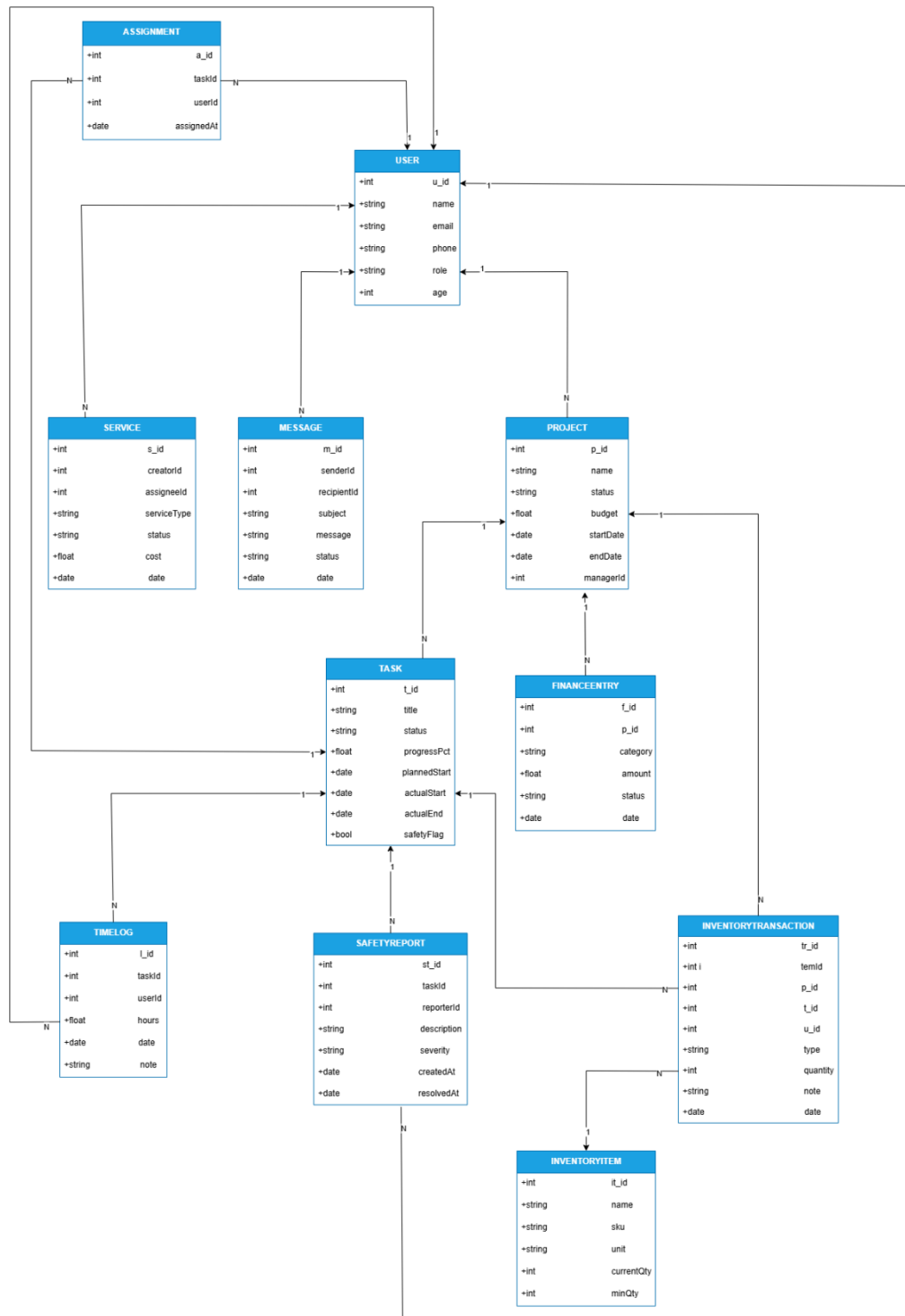
01. Design a comprehensive ER diagram to represent the data model, capturing all entities, attributes, and relationships.



2. Normalize the database schema to eliminate redundancy, improve data integrity, and ensure optimal performance.



3. Build the database using the chosen technology, ensuring adherence to the designed schema, and incorporating all necessary constraints, indexes, and relationships.



User model

```
JS UserModel.js •
Backend > Model > JS UserModel.js > [0] userSchema
1  const mongoose = require("mongoose");
2  const bcrypt = require("bcryptjs");
3  const Schema = mongoose.Schema;
4
5  const userSchema = new Schema({
6    name: {
7      type: String,
8      required: true,
9      trim: true
10   },
11   gmail: {
12     type: String,
13     required: true,
14     unique: true,
15     lowercase: true,
16     trim: true
17   },
18   phone: {
19     type: String,
20     required: false,
21     trim: true
22   },
23   role: {
24     type: String,
25     required: true,
26     enum: ["Client", "Admin", "Site Manager", "Supervisor", "Labor"],
27     default: "Client"
28   },
29   age: {
30     type: Number,
31     required: false,
32     min: 0
33   },
34   address: {
35     type: String,
36     required: false,
37     trim: true
38   },
39   password: {
40     type: String,
41     required: true,
42     minlength: 6
43   },
44 }, {
45   timestamps: true // add createdAt and updatedAt
46 });
47
48 // Hash password before saving
49 userSchema.pre('save', async function(next) {
50   if (!this.isModified('password')) return next();
51   try {
52     const salt = await bcrypt.genSalt(10);
53     this.password = await bcrypt.hash(this.password, salt);
54     next();
55   } catch (error) {
56     next(error);
57   }
58 });
59
60 module.exports = mongoose.model("User", userSchema);
61
```

Project model

```

1  const mongoose = require("mongoose");
2  const Schema = mongoose.Schema;
3
4  // Audit log schema
5  const auditlogSchema = new Schema({
6    action: {
7      type: String,
8      required: true,
9      enum: ['CREATE', 'UPDATE', 'DELETE']
10   },
11   field: String,
12   oldValue: Schema.Types.Mixed,
13   newValue: Schema.Types.Mixed,
14   timestamp: {
15     type: Date,
16     default: Date.now
17   },
18   user: String,
19   ipAddress: String
20 });
21
22 // Custom validation functions
23 const validateSpecialCharacters = function(value) {
24   if (typeof value === 'string') {
25     const specialChars = /[!#$%&']/;
26     return !specialChars.test(value);
27   }
28   return true;
29 };
30
31 const validateAge = function(value) {
32   if (value && typeof value === 'number') {
33     return value >= 0;
34   }
35   return true;
36 };
37
38 const validateBudget = function(value) {
39   if (value && typeof value === 'number') {
40     // Restrict to reasonable construction project budgets (100K to 1B)
41     return value >= 100000 && value <= 1000000000;
42   }
43   return true;
44 };
45
46 const validateDateFromToday = function(value) {
47   if (value) {
48     const today = new Date();
49     today.setHours(0, 0, 0, 0);
50     return value >= today;
51   }
52   return true;
53 };
54
55 const validateBankAccount = function(value) {
56   if (value && typeof value === 'string') {
57     // Only allow numbers for bank account
58     return /^[0-9]+$/.test(value);
59   }
60   return true;
61 };
62
63 const projectSchema = new Schema({
64   // Custom ID generation - more specific than random
65   projectId: {
66     type: String,
67     unique: true,
68     required: true,
69     default: function() {
70       const year = new Date().getFullYear();
71       const month = String(new Date().getMonth() + 1).padStart(2, '0');
72       const random = Math.floor(Math.random() * 10000).toString().padStart(4, '0');
73       return `AD-${year}${month}-${random}`;
74     }
75   },
76   name: {
77     type: String,
78     required: [true, 'Project name is required'],
79     trim: true,
80     minlength: [3, 'Project name must be at least 3 characters'],
81     maxlength: [100, 'Project name cannot exceed 100 characters'],
82     validate: {
83       validator: validateSpecialCharacters,
84       message: 'Project name cannot contain special characters (!, #, $, %)'
85     }
86   },
87   client: {
88     type: String,
89     required: [true, 'Client name is required'],
90     trim: true,
91     minlength: [2, 'Client name must be at least 2 characters'],
92     maxlength: [100, 'Client name cannot exceed 100 characters'],
93     validate: {
94       validator: validateSpecialCharacters,
95       message: 'Client name cannot contain special characters (!, #, $, %)'
96     }
97   },
98   clientContact: {
99     phone: {
100       type: String,
101       validate: {
102         validator: function(v) {
103           return !v || /^[0-9]{10}$/.test(v);
104         }
105       }
106     },
107     message: 'Phone number can only contain digits, spaces, hyphens, plus signs, and parentheses'
108   },
109   email: {
110     type: String,
111     validate: {
112       validator: function(v) {
113         return !v || /^[a-zA-Z0-9]{1,50}@[a-zA-Z0-9]{1,50}\.([a-zA-Z]{2,5})$/.test(v);
114       },
115       message: 'Please enter a valid email address'
116     }
117   },
118   bankAccount: {
119     type: String,
120     validate: {
121       validator: validateBankAccount,
122       message: 'Bank account number can only contain digits'
123     }
124   },
125   status: {
126     type: String,
127     required: true,
128     enum: {
129       values: ['Planning', 'In Progress', 'Completed', 'On Hold', 'Cancelled'],
130       message: 'Status must be one of: Planning, In Progress, Completed, On Hold, Cancelled'
131     },
132     default: 'Planning'
133   },
134   priority: {
135     type: String,
136     enum: ['Low', 'Medium', 'High', 'Critical'],
137     default: 'Medium'
138   },
139   startDate: {
140     type: Date,
141     required: [true, 'Start date is required'],
142     validate: {
143       validator: function(value) {
144         // Only enforce future-or-today constraint when creating a new document
145         if (this.isNew) {
146           const today = new Date();
147           today.setHours(0, 0, 0, 0);
148           return value >= today;
149         }
150         // On edits, allow retaining or setting past dates
151         return true;
152       },
153       message: 'Start date must be from today onwards for new projects'
154     }
155   },
156   endDate: {
157     type: Date,
158     required: [true, 'End date is required'],
159     validate: {
160       validator: function(value) {
161         if (!value) return false;
162         const today = new Date();
163         today.setHours(0, 0, 0, 0);
164         const isFromTodayOnwards = value >= today;
165         if (!this.startDate) return isFromTodayOnwards;
166         return isFromTodayOnwards && value > this.startDate;
167       }
168     }
169   },
170   budget: {
171     type: Number,
172     required: [true, 'Budget is required'],
173     min: [100000, 'Budget must be at least Rs. 100,000'],
174     max: [1000000000, 'Budget cannot exceed Rs. 1,000,000,000'],
175     validate: {
176       validator: validateBudget,
177       message: 'Budget must be between Rs. 100K and Rs. 1B'
178     }
179   },
180   completion: {
181     type: Number,
182     required: true,
183     min: [0, 'Completion cannot be less than 0%'],
184     max: [100, 'Completion cannot exceed 100%'],
185     default: 0
186   },
187   // Project team information
188   projectManager: {
189     name: {
190       type: String,
191       trim: true,
192       validate: {
193         validator: validateSpecialCharacters,
194         message: 'Project manager name cannot contain special characters (!, #, $, %)'
195       }
196     }
197   }
198 });

```

```

206   }
207 },
208 age: {
209   type: Number,
210   validate: {
211     validator: validateAge,
212     message: 'Age must be a positive number'
213   }
214 },
215 experience: {
216   type: Number,
217   min: [0, 'Experience cannot be negative']
218 },
219 },
220
221 // Project details
222 description: {
223   type: String,
224   maxlength: [1000, 'Description cannot exceed 1000 characters'],
225   validate: {
226     validator: validateSpecialCharacters,
227     message: 'Description cannot contain special characters (!, #, $, %)'
228   }
229 },
230
231 location: {
232   address: {
233     type: String,
234     trim: true,
235     validate: {
236       validator: validateSpecialCharacters,
237       message: 'Address cannot contain special characters (!, #, $, %)'
238     }
239   },
240   city: {
241     type: String,
242     trim: true,
243     validate: {
244       validator: validateSpecialCharacters,
245       message: 'City cannot contain special characters (!, #, $, %)'
246     }
247   },
248   coordinates: {
249     latitude: Number,
250     longitude: Number
251   }
252 },
253
254 // Timestamps
255 createdAt: {
256   type: Date,
257   default: Date.now
258 },
259 updatedAt: {
260   type: Date,
261   default: Date.now
262 },
263
264 // Audit trail
265 auditlog: [auditlogSchema]
266 });
267
268 // Pre-save middleware to update timestamps and add audit log
269 projectSchema.pre('save', function(next) {
270   this.updatedAt = new Date();
271
272   // Add audit log for updates
273   if (this.isModified() && !this.isNew) {
274     this.auditlog.push({
275       action: 'UPDATE',
276       timestamp: new Date(),
277       field: 'general',
278       newValue: 'Project updated'
279     });
280   }
281   next();
282 });
283
284 // Pre-remove middleware to add audit log
285 projectSchema.pre('remove', function(next) {
286   this.auditlog.push({
287     action: 'DELETE',
288     timestamp: new Date(),
289     field: 'general',
290     oldValue: 'Project deleted'
291   });
292   next();
293 });
294
295 // Static method to get project statistics (60-30-10 rule)
296 projectSchema.statics.getProjectStats = function() {
297   return this.aggregate([
298     {
299       $group: {
300         _id: '$status',
301         count: { $sum: 1 }
302       }
303     }
304   ]),
305
306   {
307     $group: {
308       _id: null,
309       total: { $sum: '$count' },
310       statusCounts: {
311         $push: {
312           status: '$_id',
313           count: '$count'
314         }
315       }
316     },
317     {
318       $project: {
319         _id: 0,
320         total: 1,
321         statusCounts: 1,
322         distribution: {
323           $map: {
324             input: '$statusCounts',
325             as: 'status',
326             in: {
327               status: '$$status.status',
328               count: '$$status.count',
329               percentage: {
330                 $multiply: [
331                   { $divide: ['$status.count', '$total'] },
332                   100
333                 ]
334               }
335             }
336           }
337         }
338       }
339     }
340   ]);
341
342   // Instance method to add audit log entry
343   projectSchema.methods.addAuditLog = function(action, field, oldValue, newValue, user, ipAddress) {
344     // Initialize auditlog array if it doesn't exist
345     if (!this.auditlog) {
346       this.auditlog = [];
347     }
348
349     this.auditlog.push({
350       action,
351       field,
352       oldValue,
353       newValue,
354       timestamp: new Date(),
355       user,
356       ipAddress
357     });
358     return this.save();
359   };
360
361   module.exports = mongoose.model('Project', projectSchema);
362 }

```

Inventory model

```

JS InventoryItem.js JS BuyerInventoryModels X
Backend > Model > JS BuyerInventoryModels.js > METRIC_ENUM
1 // Model/BuyerInventoryItem.js
2 const mongoose = require("mongoose");
3 const { Schema } = mongoose;
4
5 // Enumerations aligned with UI and InventoryItem model
6 const TYPE_ENUM = [
7   "Cement",
8   "Granite",
9   "Sand",
10  "Concrete Blocks",
11  "Steel Bars",
12  "Bricks",
13  "Tiles",
14  "Paint",
15  "Other Construction Material",
16 ];
17 const METRIC_ENUM = [
18   "Bags",
19   "Tons",
20   "Cubes",
21   "Packets",
22   "Count",
23   "Liters",
24   "Pieces",
25   "Kg",
26   "Other",
27 ];
28
29 const LineItemSchema = new Schema(
30 {
31   type: {
32     type: String,
33     required: true,
34     enum: TYPE_ENUM,
35   },
36   metric: {
37     type: String,
38     required: true,
39   },
40   amount: {
41     type: Number,
42     required: true,
43     min: 0,
44     default: 0,
45   },
46   unitPrice: {
47     type: Number,
48     required: true,
49     min: 0,
50     default: 0,
51   },
52   seller: {
53     type: String,
54     trim: true,
55     default: "",
56   },
57   lineTotal: {
58     type: Number,
59     required: true,
60     min: 0,
61     default: 0,
62   },
63 },
64 { _id: false }
65 );
66
67 const buyerOrdersSchema = new Schema(
68 {
69   customerName: {
70     type: String,
71     required: true,
72     trim: true,
73     maxlength: 120,
74   },
75   currency: {
76     type: String,
77     default: "USD",
78     uppercase: true,
79     trim: true,
80   },
81   items: {
82     type: [LineItemSchema],
83     validate: [(arr) => arr && arr.length > 0, "At least one line item is required"],
84     default: [],
85   },
86   orderTotal: {
87     type: Number,
88     min: 0,
89     default: 0,
90   },
91 },
92 { timestamps: true }
93 );
94
95 function computeOrderTotals(next) {
96   // compute each lineTotal and overall orderTotal
97   if (Array.isArray(this.items)) {
98     let sum = 0;
99     this.items = this.items.map((li) => {
100       const qty = Number(li.amount || 0);
101       const price = Number(li.unitPrice || 0);
102       const lt = Math.max(0, qty * price);
103       return { ...(li.toObject?.() ?? li), lineTotal: lt };
104     });
105     sum = this.items.reduce((acc, li) => acc + Number(li.lineTotal || 0), 0);
106     this.orderTotal = Math.max(0, sum);
107   } else {
108     this.orderTotal = 0;
109   }
110   next && next();
111 }
112
113 buyerOrderSchema.pre("validate", computeOrderTotals);
114 buyerOrderSchema.pre("save", computeOrderTotals);
115
116 module.exports = mongoose.model("BuyerInventoryItem", buyerOrderSchema);
117

```


Communication model

```
JS ServiceRoute.js M JS MessageModel.js X JS Service.js M
Backend > Model > JS MessageModel.js > ...
1  const mongoose = require('mongoose');
2
3  const MessageSchema = new mongoose.Schema({
4    subject: { type: String, required: true, trim: true, minlength: 3, maxlength: 100 },
5    sender: { type: String, required: true, trim: true, minlength: 2, maxlength: 50 },
6    recipient: { type: String, required: true, trim: true, minlength: 2, maxlength: 50 },
7    // New: store actual user ids to route by login
8    senderId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
9    recipientId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
10   message: { type: String, required: true, trim: true, minlength: 1, maxlength: 1000 },
11   date: { type: Date, default: Date.now },
12   status: { type: String, enum: ['Unread', 'Read', 'Archived'], default: 'Unread' },
13   // Read tracking
14   isRead: { type: Boolean, default: false },
15   readAt: { type: Date, default: null }
16 });
17
18 module.exports = mongoose.model('Message', MessageSchema);
19 |
```

Finance model

```
JS FinanceModel.js X
Backend > Model > JS FinanceModel.js > ...
1 const mongoose = require("mongoose");
2 const Schema = mongoose.Schema;
3
4 const financeSchema = new Schema({
5   description: { type: String, required: true },
6   Project_Name: { type: String, required: true },
7   category: { type: String, required: true },
8   amount: { type: Number, required: true },
9   date: { type: Date, default: Date.now },
10  status: { type: String, required: true, default: "Unpaid" },
11  project: { type: Schema.Types.ObjectId, ref: "Project" },
12  bankSlipPath: { type: String },
13  stage: {
14    type: String,
15    enum: ["deposit", "balance", "full"],
16    default: "full"
17  },
18  expectedAmount: { type: Number },
19  paidAmount: { type: Number },
20  paidAt: { type: Date },
21  submittedBy: { type: Schema.Types.ObjectId, ref: "User" },
22  source: {
23    type: String,
24    enum: ["admin", "client", "system"],
25    default: "admin"
26  }
27 });
28
29 module.exports = mongoose.model("FinanceModel", financeSchema)
```